CONFIDENTIAL — INVESTOR TECHNICAL BRIEF

Concept2Cure.RI — Investor Technical Brief

Version · 2026-09-08
Codebase branch · `concept2cure-v2` (the sole production branch)
Audience · technical diligence readers and their principals

PREPARED FOR PROSPECTIVE INVESTORS

Table of Contents

---

1. Executive summary

Concept2Cure.RI is a regulated-industry operating system for life-sciences submission work. It replaces the eCTD authoring silo, the regulatory intelligence silo, the QMS/audit silo, and the shared-drive-plus-Word workflow with a single, governed, memory-carrying environment in which the AI is a first-class participant — but never an unchecked one.

Where competitors sell either an AI chat wrapper over a document store, or a static submission tool with a chatbot bolted on, we have built the missing middle: a **governance kernel** that decides which action is allowed, an **AI gateway** that routes work across Claude, GPT-4, and fallbacks with full audit, an **intelligence stack** (RIM, CORTEX Prime, Precedent Engine, Submission Twin) that reasons across the submission as a graph rather than a folder, and a **three-layer memory** that carries what the team has decided across sessions and users.

The verified size of what we have built:

METRIC	VALUE
Server TypeScript files	4,180
Client React files (`.tsx`)	439
HTTP route modules (top level)	368
Service files (recursive)	2,956 across 222 service directories
Database migration files	554 (247 in `migrations/`, 307 in `db/migrations/`)
`pgTable` declarations across schema files	~694
Multi-tenant scoping references (`organizationId`) in `shared/schema.ts` alone	736
Automated test files (`.test.ts`/`.test.tsx`/`.spec.ts`)	2,348 total, 448 in `tests/`, 1,522 under `server/`
Docs `.md` files under `docs/`	672 across 55 subdirectories

Every one of those numbers is grounded in a specific path we can walk a diligence reader to, and §20 gives the exact command behind each. They were measured on 2026-09-08 against `concept2cure-v2`; the repository took 48 commits that day alone, so expect the counts to have grown by the time you re-run them. Re-run them — that is the point of publishing the commands.

---

2. What the product is, and what it replaces

Concept2Cure.RI is the operating system for regulated submission work at drug, device, and diagnostic companies. It is not a chatbot. It is not a document store. It is an environment in which:

- Every artifact is versioned, provenance-linked, and traceable to source evidence.
- Every AI action is routed through a policy kernel that decides *allow / review / deny* before the action reaches the user.
- Every governed action (approve, submit, promote, e-sign) captures reason-for-change, signer identity, TOTP re-authentication, and an HMAC-sealed audit event.
- Every generation is grounded in a retrieval pass across the tenant's own evidence, and the resulting draft carries sentence-level provenance the reviewer can follow to source.
- Every project accumulates a memory: locked facts, decisions, open questions, next actions, artifacts already produced — reusable across sessions and users without re-briefing the AI.

What it replaces, workstream by workstream:

- **Veeva Vault / MasterControl** for authoring and QMS.
- **DXC / Certara / Lorenz** for eCTD lifecycle and submission gateway management.
- **Cortellis / Citeline** as the regulatory intelligence lookup tool, superseded by a stack that reads *your* project's evidence against the regulator corpus and gives you a next best action rather than a search result.
- The **spreadsheet-and-Word workflow** that most small sponsors still use for pre-IND, pre-sub, and CRL response drafting — collapsed into one governed surface.

The product is used before, during, and after submission. Before: gap register, precedent mining, protocol design. During: co-authored eCTD Modules 1–5, section-level readiness, reviewer-simulation adversarial

review. After: PSUR/DSUR periodic reporting, CAPA, deficiency response, RIM registration and label lifecycle.

---

3. The regulated use cases we address

The platform ships against nine primary use cases, each of which is a full workstream (not a demo), each of which has its own golden-journey test suite committed to the repo, and each of which is a distinct commercial anchor.

3.1 IND / NDA / BLA authoring (drug). The user opens a program, ingests their nonclinical, CMC, and clinical data, and the platform composes IND Module 2 summaries and Module 3 CMC narratives with sentence-level provenance to source. Golden journey: `tests/golden-journeys/drug-nda-ectd.journey.test.ts`, `ind-authoring.journey.test.ts`.

3.2 510(k) / De Novo authoring (device). Predicate mining across `predicate.fda_510k_clearances` and `predicate.fda_510k_embeddings`, substantial-equivalence narrative generation, eSTAR-compliant packaging. Golden journey: `tests/golden-journeys/device-510k-estar.journey.test.ts`.

3.3 CER for EU MDR / IVDR. Clinical Evaluation Report authoring against MDCG 2020-13, with living evidence spine and post-market update flow. Golden journey: `tests/golden-journeys/cer-eu-mdr.journey.test.ts`. Dedicated IVDR router: `server/routes/ivdr-routes.ts` (72 KB).

3.4 CMC Module 3 co-authoring. Manufacturing narrative generation from process, spec, and stability data — validated by the CMC dataroom review `docs/reports/CMC_MODULE3_DATAROOM_VALIDATION_2026-08-23.md`.

3.5 CSR (Clinical Study Report) generation. ICH E3-compliant CSR builder at `server/services/csr-builder.ts` (853 lines). Draws on `csr_reports` / `csr_details` and the CSR knowledge database at `shared/schema/csr-knowledge-db.ts` (48.9 KB).

3.6 Deficiency and CRL response. The AnA RI orchestrator (`server/routes/ana-ri.ts`) surfaces a live deficiency taxonomy and evaluation rubric. Every response is scored against a deterministic-plus-LLM rubric before it reaches the user.

3.7 Regulatory intelligence and horizon scanning. RIM (Regulatory Intelligence Model) at `server/services/intelligence/` ingests guidance, precedent, and reviewer signals; the guidance-impact scanner (`guidance-impact-scanner.ts`) tells you which of your in-flight artifacts a new FDA guidance affects.

3.8 Post-market surveillance (PSUR, DSUR, PMS). `shared/schema/gspr-postmarket.ts`, `capa-mdr.ts`, and the periodic-report pathways in the Intelligent Report Engine (12 report domains, 15 regulatory bodies).

3.9 Regulatory operations at scale. RIM product/registration/label lifecycle (`shared/schema/rim.ts`), submission gateway management (`server/services/submission-gateways/`), CAPA, inspection readiness (`shared/schema/inspection.ts`), IACUC/IBC/IRB (`shared/schema/iacuc.ts`, `ibc.ts`, `irb.ts`).

The commercial reality: most competitors do one of these; **we do all nine, on one governance kernel, sharing one memory, one audit chain, one design system.**

---

4. The nine end-to-end workstreams that ship

The nine use cases above map onto nine engineered workstreams, each with its own routes, services, schema domain, and tests. They are not features — they are wall-to-wall vertical slices:

1. **Program & Charter** — `shared/schema/project-charter.ts` (38 KB), `programs.ts` (21 KB). A program has objectives, target markets, target regulators (up to 15 supported bodies), and a chartered plan.
2. **Evidence Ingestion & Vault** — `server/services/vault/`. Documents are ingested, chunked (`document-chunking.service.ts`), embedded, and made available for retrieval and lineage.
3. **Authoring** — `server/routes/authoring.router.ts` (168 KB), `authoring-actions.ts` (132 KB), `documentAuthoring.routes.ts` (58 KB). Rich-text co-authoring with span-level lineage (`shared/schema/document-span-lineage.ts`) so every accepted machine sentence is traceable.
4. **Review & Adversarial Simulation** — `shared/schema/reviewer-simulation.ts`, `shadow-review.ts`. We simulate the regulator on your draft before you submit.
5. **Governed Actions & Sign-Off** — `server/services/ana-ri/governed-action-signoff.ts`, `part11-governance.ts`. Reason-for-change, TOTP re-auth, signature-integrity check.
6. **Submission Assembly & Transmission** — `server/services/submission-gateways/governed-transmit.ts`. eCTD/eSTAR packaging and gateway-agnostic transmission.
7. **RIM & Post-Market Lifecycle** — product/registration/label state machines in `shared/schema/rim.ts` plus the post-market schema.
8. **Audit, Compliance & Attestation** — `server/services/audit/` (10+ services), signed audit exports, chain-integrity sweep job.
9. **Analytics & Reporting** — the Intelligent Report Engine (3,562 lines, 12 report domains) plus the Submission Twin (1,430 lines).

Each workstream shares four cross-cutting services: **AI Gateway**, **Kernel** (policy + decisions), **Memory** (working/client/project), and **Precedent Engine**. That shared core is why we can add a tenth workstream (early: gene-therapy CMC, ATMP) in weeks, not quarters.

---

5. System architecture at a glance

At the outermost layer, the runtime is a hardened Express 5 server (Node 22, pinned `>=22.0.0 <23.0.0`) fronting a React 19 client, with a Socket.io real-time channel for collaborative editing and cursor presence, and a Bull-on-Redis queue for long-running AI actions and export jobs. Every browser request lands on a rate-limited, helmeted, tenant-scoped route.

At the next layer, every request that reads or writes governed data flows through:

- **Tenant scope establishment** — `server/middleware/establishRequestTenantScope.ts` opens a Postgres session with the verified JWT's `organizationId` bound, enabling row-level security. `RLS_ENFORCE=on` is fail-closed: any query issued without an active scope is refused.
- **The Kernel** — `server/src/control-plane/kernel.ts` plus seven `server/services/kernel-*` files. Every consequential action is evaluated for governance / security / observability signals and receives a decision (`allow` | `review` | `deny`) with a rationale, a regulatory reference, and an evidence hash.

Decisions are recorded to `ai_kernel_decision_records`, immutably chained (`20260325_ana_kernel_log_immutability_hashchain.sql`).

- **The AI Gateway** — `server/services/ai-gateway/gateway.ts`. All model calls route through it. Providers, fallbacks, retries, and cost telemetry are handled once, in one place.

Below that sit the intelligence services (RIM, CORTEX Prime, Precedent Engine, Foresight — see §9), the memory services (§10), and the authoring services. Underneath is Neon Postgres with `pgvector` for semantic retrieval and per-row multi-tenant isolation.

The client is a single-page application built with Vite, TanStack Query for server state, and a governed component library. The shell — `client/src/concept2cure/v2/V2App.tsx` with the chrome in `Shell.tsx` (Rail, TopBar, AnaRail, CmdK) — provides the environment; every module is a surface registered in `config/ui-surface-registry.json`.

---

6. The control plane: kernel, decisions, and adaptive policy

The kernel is the piece an investor should look at first, because it is the piece competitors cannot casually replicate. A chat wrapper can be rebuilt in a weekend; a policy kernel that has been proven against real submission traffic cannot.

What the kernel actually is. It is a set of pure functions plus a persistence layer that evaluate every governed action against three domains:

- `governance` — is this action allowed under the tenant's regulatory posture and role model?
- `security` — does this action need step-up authentication, and is the requester's context still valid?
- `observability` — is this action being traced, and does the trace carry enough evidence to reconstruct it later?

Each domain returns a `KernelDecision` of `allow` | `review` | `deny`, a rationale string, an ISO timestamp, an optional regulatory reference (e.g. `21 CFR §11.10(e)`), and an evidence payload. The traces compose into a `KernelTraceStep[]` that lives on the request.

The eight kernel services:

FILE	ROLE
`kernel-decision-record.ts`	Immutable record of every governance decision made for a request
`kernel-router.ts`	Chooses task type, routing strategy, and token budget for the request
`kernel-goal-planner.ts`	Generates explicit multi-step `GoalPlans` with dependencies, success criteria, and replan triggers
`kernel-adaptive-policy.ts`	Tenant/feature-tier scoped policy bundles that version over time
`kernel-observability.ts`	Trace and metric emission for kernel decisions
`kernel-beta-readiness.ts`	Feature-flag readiness gates so we ship the same binary to beta and GA
`kernel-plan-runtime.ts`	Executes goal plans and reconciles the actual outcome against the planned success criteria
`kernel-agent-protocol.ts`	The contract agents speak to the kernel over, and the protocol events recorded against it

Why this matters commercially. Regulated buyers cannot ship AI they cannot audit. They need to answer three questions to their own quality organization: *what did the AI do, what said it could, and what will let us prove that in an inspection*? The kernel's decision records are that answer — signed, chained, exportable —

and they are the artifact that makes the platform inspectable. Systems without a kernel of this shape will fail the inspection question, whatever else they can demonstrate.

---

7. The AI Gateway and multi-provider routing

Every large language model call from anywhere in the platform is issued through `server/services/ai-gateway/gateway.ts`. There is no alternative code path — a bypass would fail lint, would fail our design-CI gates, and would fail its own kernel decision.

Providers, in order of quality-weighted preference:

- **Anthropic** — `claude-opus-4` (200K context, quality score 99 internally), `claude-sonnet-4` (97), `claude-haiku-4` (80).
- **OpenAI** — `gpt-4o` (128K, quality 95), `gpt-4o-mini` (82).
- **Kimi** — available as tertiary fallback.

Fallback and retry. The gateway does not simply retry on failure — it consults `getFallbackModels()` for the task type (drafting, evaluation, extraction, embedding), and it walks the ladder. Retries use exponential backoff with jitter (base delay × 2^attempt + 0–30% jitter). Non-transient errors (400, 401, 403) abort immediately rather than burning budget. A final deterministic-demo mode returns canned responses when no key is available, so that unit tests and dev environments do not depend on live vendors.

What is measured. `GatewayAuditLogger` persists every request and response to the database; `GatewayPolicyEngine` enforces policy before dispatch; per-provider health is tracked with `recordSuccess()` / `recordFailure()` so health-aware routing can down-weight a struggling provider before it degrades user experience. Structured logs flow through `createScopedLogger('ai-gateway')`.

Why this matters commercially. The AI Gateway is the difference between "we use Claude" and "we are vendor-independent." Enterprise buyers assume providers will fail, get expensive, or change terms. Our answer is: our runtime does not care which model served your last request. Their evidence lives in our schema, not in a vendor's chat history.

---

8. Retrieval: the 9-step provenance-tracked RAG pipeline

The chat surface is not a chat surface. It is a nine-step retrieval and generation pipeline whose intermediate artifacts are auditable. The router is thin (`server/routes/chat.ts`, 99 lines); the pipeline lives in `server/routes/chat/` split across `send-message.ts`, `stream.ts`, `threads.ts`, `upload.ts`, `provenance.ts`, `verifier.ts`, and `shared.ts`.

The nine steps, at high level:

1. **Intent classification** via `IntentLens` — regulatory role and task inference.
2. **Tenant-scoped hybrid retrieval** across the vault (`document-chunking.service.ts`, `document-catalog-search.ts`) using `pgvector` similarity plus keyword blend (`rag-fusion.ts`).
3. **Memory retrieval** — the three-layer memory system fetches working / client / project memory in parallel with per-layer timeouts (`memory-context-assembler.ts`, `memory-orchestrator.ts`).

4. **Precedent injection** — the Precedent Engine returns comparable outcomes and adversarial history for the artifact under discussion.
5. **Context assembly** — `lumen-context-builder.ts` (1,090 lines) composes the system prompt from project context, document context, workflow context, and conversation context, layered onto the static `REGULATORY_SYSTEM_PROMPT`.
6. **Kernel review** — the constructed request is scored by the kernel; low-risk actions proceed, high-risk actions escalate.
7. **Gateway dispatch** — the AI Gateway routes the call to the appropriate model with retry, fallback, and telemetry.
8. **Verification** — `verifier.ts` runs deterministic checks (citation presence, refusal patterns, hallucination flags) and, for governed drafts, sentence-level lineage attribution.
9. **Provenance seal** — `provenance.ts` records inputs, outputs, model, decision trace, and hash into the audit chain.

Every one of those steps is testable in isolation and re-runnable on a stored record. That is what lets us claim inspection-readiness, not merely accuracy.

Embeddings. We are pluralist by design. `pgvector` columns exist at three dimensions:

- `1536` — OpenAI `text-embedding-3-small`, our default for text and memory (`working_memory_embeddings`, `vault_document_chunks`, `precedent_engine`, and eight other tables).
- `3072` — OpenAI `text-embedding-3-large`, used for high-value atoms (Cortex Prime, higher-fidelity precedent).
- `1024` — Cohere / Voyage / BGE — enabled for tenants that require a non-OpenAI embedding lineage.

A historical HNSW-index dimension cap (2,000) forced us to migrate `lumen_data_atoms.embedding` down from 3072 to 1536 — `db/migrations/20260730_fix_atom_embedding_dimension.sql`. We keep both dimensions on CORTEX atoms (`embedding1536` + `embedding3072`) so a tenant can flip embedding models without re-ingesting.

---

9. The intelligence layer: RIM, CORTEX Prime, Precedent Engine

The intelligence layer is what turns a document store into a submission strategist. It is three services — plus a fourth (Foresight) that we intentionally retired because the shape it took at first was wrong for the domain — and one report engine.

9.1 RIM — the Regulatory Intelligence Model

Path: `server/services/intelligence/` — 37 TypeScript modules.

- **Central orchestrator** `rim.ts` runs at `RIM_VERSION = '1.1.0'`. Every run is stamped with a run ID, the RIM version, and the pattern version, so a decision made six months ago can be exactly re-derived today. Runs that fail to persist are marked `degraded`, not silently swallowed.
- **Judgment framework** `judgment-framework.ts` at `JUDGMENT_FRAMEWORK_VERSION = '1.2.0'` computes six explicit judgment models: Evidence Sufficiency, Defensibility, Reviewer Sensitivity, Claim Risk, Cross-Section Consistency, Submission Risk. Each is a weighted composite of deterministic checks, heuristic

checks, and LLM-assisted scoring, with a `MODEL_WEIGHTS` map (for example: 0.30 / 0.25 / 0.25 / 0.20 for Evidence Sufficiency).

- **Pattern registry** `pattern-registry.ts` at `PATTERN_REGISTRY_VERSION = '1.3.0'` codifies eight categories (`deficiency` , `reviewer_trigger` , `rejection` , `strong_language` , `weak_language` , `data_gap` , `consistency_issue` , `formatting`) with seeded patterns that grow with the corpus.
- Surrounding modules: readiness scoring, recommendation engine, evidence confidence, cross-module intelligence, cross-artifact consistency scanner, learning loop, next-best-action engine, risk model + backtest, precedent mining, outcome-precedent ingestor, guidance impact scanner, calibration, counterfactual replay.

RIM is not a chatbot personality — it is a set of composable scoring functions. Every score has a versioned weight, an evidence trail, and a replay path.

9.2 CORTEX Prime — the knowledge graph

Path: `server/services/cortexPrimeService.ts` — 1,207 lines. Five primitives:

- `CortexAtom` — a single knowledge unit; carries dual embeddings (1536 and 3072), a quality score, and structured metadata.
- `CortexEdge` — typed, weighted, directional relations between atoms.
- `CortexAgent` — a scoped agent identity that reads and writes into the graph.
- `CortexThread` — a conversation that operates over a bounded set of context atoms (`contextAtomIds[]`).
- `CortexTrace` — an audit trail over agent operations on the graph.

CORTEX Prime exposes graph traversal — `startAtomId` , `edgeTypes` , `maxDepth` , `minStrength` — which is what lets us do things like "find every precedent atom within three edges of this claim, weighted by edge confidence." This is the technical substrate under things a competitor cannot: cross-artifact consistency across ten years of a sponsor's history, precedent-aware CRL response drafting, and change-impact analysis when a new guidance drops.

9.3 The Precedent Engine

Path: `server/services/precedent-engine.ts` — 2,081 lines. Nine operations:

`precedent.search` , `precedent.compare` , `precedent.risk` , `precedent.strategy` , `authoring.check` , `precedent.crlTriggers` , `precedent.rtfTriggers` , `precedent.emaPatterns` , `precedent.advisoryRisk` .

Backing tables: `predicate.fda_510k_clearances` , `predicate.fda_510k_embeddings` , `predicate.predicate_lineage` , `predicate.predicate_safety_signals` , `adversarial.regulatory_adversarial_precedents` , `precedent.regulatory_precedents` , plus the CSR knowledge tables. Tenant-scoped through `precedent-isolation.ts` .

This is the piece prospects underweight and then find they cannot live without. The moment a sponsor sees their next 510(k) claim scored against every predicate with a similar substantial-equivalence pattern, and their CRL trigger risk called out sentence by sentence, the ROI conversation is over.

9.4 Foresight — what we removed and why

`server/services/foresight/foresight-ai-engine.ts` does not exist as a live service. The foresight prediction tables were dropped intentionally

(`db/migrations/20260725_drop_orphaned_foresight_prediction_tables.sql`) after a design review concluded that a general-purpose "predict outcomes" service was the wrong abstraction — it belonged inside the specific judgment models in RIM, tied to specific artifacts and specific regulators, not as a standalone service. Removing it was a correctness decision. The learnings landed in `learning-loop-service.ts` and `outcome-feature-extraction.ts` inside the intelligence layer.

We document what we retire. Retirement decisions are as much a signal to an investor as ship decisions.

---

10. The three-layer memory system

Everything a competitor does with "remember my chat" we do with a memory system that separates three lifetimes and reconciles them.

Layer 1 — Working memory. `server/services/working-memory.ts`. Structured summaries per conversation with fixed sections: Objective, Locked Facts, Decisions, Open Questions, Next Actions, Created Artifacts. A rolling threshold (`WORKING_MEMORY_THRESHOLD = 20` messages) triggers compaction. Optional semantic search over working memory via `text-embedding-3-small`, gated by `ENABLE_SEMANTIC_WORKING_MEMORY`. This is what lets a user leave a conversation on Thursday and pick up exactly where they left off on Monday.

Layer 2 — Client intelligence memory. `server/services/client-intelligence-memory.ts`. Sponsor-scoped memory: their standards, their preferred vendors, their prior regulator feedback, their historical CRL/RTF triggers. Every entry has a lifecycle — `status: 'active'` — and can be `superseded` (with a pointer to the successor entry) rather than deleted, so the audit trail is preserved. Retrieval defaults to `includeSuperseded: false`.

Layer 3 — Project intelligence memory. Same store, scoped to a single project or program. Backing tables live in the monolith schema at `projectIntelligenceProfiles`, `projectMemoryEntries`, and `projectIngestedDocuments`. Every project accumulates its own decisions, facts, and open questions — reusable across all users on the project, and across all AI sessions in that project.

The orchestrator. `server/services/memory-orchestrator.ts` coordinates the three layers into a single ranked, deduplicated list. Ranking formula per layer:

score = priorityBoost + (similarity × w_sim) + (confidence × w_conf) + verifiedBoost

`memory-context-assembler.ts` (422 lines) fetches all three layers in parallel with per-layer timeouts so a memory outage in one layer cannot stall the pipeline, and normalizes into the `SharedMemoryContract` — the interface the rest of the platform reads.

Why this matters. In regulated work, "the AI forgets what I told it yesterday" is not a UX complaint — it is a compliance risk. The memory system, with its supersession lifecycle and hash-chained audit, is what lets us claim the AI is a durable collaborator, not a session.

---

11. Submission Twin and the Intelligent Report Engine

Two services that only make sense once the layers above exist:

Submission Twin — `server/services/submission-twin-service.ts`, 1,430 lines. It keeps a live in-database twin of your regulatory submission with six capabilities:

1. **Claim-to-evidence integrity map** — every claim in every section points to the evidence atoms that support it. Break the link, we alert you.
2. **Narrative drift detection** — when the CMC section quietly diverges from the clinical section on the same fact, we catch it before the regulator does.
3. **Regulator challenge simulation** — the twin is asked the questions a reviewer would ask, using the reviewer-simulation model.
4. **Change consequence intelligence** — if you edit a spec on page 3 of Module 3, the twin tells you every downstream section, in every module, that must be re-verified.
5. **Next best artifact prediction** — given the current state of the twin, what is the single most valuable next thing to produce?
6. **Readiness and fragility modeling** — a submission is only as strong as its weakest section; the twin identifies which sections would collapse under the smallest change.

Tables: `submissionTwinClaims`, `submissionTwinEvidenceLinks`, `submissionTwinDriftAlerts`, `submissionTwinChallenges`, `submissionTwinChangeImpacts`, `submissionTwinAssessments`.

Intelligent Report Engine — `server/services/intelligent-report-engine.ts`, 3,562 lines. Twelve report domains (`regulatory_submission`, `clinical_study`, `cmc_manufacturing`, `pharmacovigilance`, `quality_management`, `compliance_attestation`, `strategic_intelligence`, `provenance_audit`, `device_regulatory`, `biostatistics`, `environmental_safety`, `cross_functional`) targeting fifteen regulatory bodies (FDA, EMA, PMDA, NMPA, TGA, Health Canada, MHRA, ANVISA, MFDS, Swissmedic, ICH, WHO PQ, CDSCO, HSA, SAHPRA). SHA-256 hash chains, Merkle roots, immutable report records, atom-level report provenance, and indemnification attestations — each intended to answer 21 CFR Part 11 §11.10(a–k).

The engine is not a template — it composes reports from atoms and evidence links in the twin, so a report is a materialized view of the truth in the system, not a hand-authored PDF that drifts.

---

12. The authoring surface and governed actions

Authoring is the most-used surface in the product, and it is also the most instrumented.

Where authoring lives. The routes are split across `server/routes/authoring.router.ts` (168 KB), `server/routes/authoring-actions.ts` (132 KB), `server/routes/documentAuthoring.routes.ts` (58 KB). The editor surface on the client is `client/src/concept2cure/v2/surfaces/DocumentAuthoring.tsx` with internals in `client/src/concept2cure/v2/editor/`.

How a governed action works.

1. The user takes an action inside the editor — accept a machine draft, promote a section, sign off on a submission package.
2. The request passes through `authoringObjectAuthorization.ts`: does the requester have access to this artifact, in this project, in this organization?
3. `authoring-actions.ts` resolves the governed context — `resolveGovernedContext()` — validating `projectId`, `artifactId`, `organizationId`, and any escalation gates set by the kernel's adaptive policy.
4. If the action requires an e-signature (approve, submit), `signature-manifestation.ts` composes the human-readable manifestation, `signing-authority.ts` verifies the signer, `mfaService.ts` requires TOTP re-authentication (RFC 6238, HMAC-SHA1, 30s, 6 digits, ±1 window, AES-256-GCM at rest).

5. The signed action is written through `governed-transmit.ts` with an HMAC seal (`audit-hmac-seal.ts`) into the hash-chained audit tables.
6. A `KernelDecisionRecord` is emitted, immutable, and readable by an inspector.

Span-level lineage. `shared/schema/document-span-lineage.ts` records, for every accepted machine sentence, the model, the retrieval set, the reviewer, and the timestamp. This is the difference between "the AI wrote this" and "here is the exact provenance of this sentence, and here are the four cited sources."

---

13. The database: schema, migrations, row-level tenancy

Database work is where the engineering discipline shows up most clearly.

Postgres on Neon, with `pgvector`. `drizzle.config.ts` targets `postgresql`, reads `DATABASE_URL_ADMIN` / `NEON_DATABASE_URL_ADMIN`, and refuses `.pooler.` URLs during migration — migration DDL must go through a direct Neon endpoint so schema drift cannot happen at the connection-pool layer. The pgvector extension is enabled at the SQL level in each migration that needs it.

Migration inventory. 554 migration files total: 247 in `migrations/` (Drizzle-numbered from `0000_sweet_joseph.sql` — a 420 KB baseline — through the dated set spanning `20260125` → `20260907`), and 307 in `db/migrations/` (a GCC-numbered core from `000_gcc_bootstrap_core.sql` → `103_load_complete_templates.sql`, plus its own dated set).

The re-execution discipline. Every migration in the set re-executes on every deploy. This is a deliberate choice, documented in `CLAUDE.md` Rule 1: the applier reads and executes every entry of `C2C_MIGRATION_FILES` unconditionally, and the drift is written to a journal. The rule that follows is a cultural artifact — *"to remove a column, constraint, or table that any file in the set creates, amend the creating migration in place. Do not append a DROP."* The rule is enforced by `npm run ci:migration-drop-safety`, and its failure branch is exercised by its own self-test (`ci:migration-drop-safety:selftest`). Genuine exceptions are baselined with a written reason.

Schema scale. `shared/schema.ts` is an 840 KB monolith with 419 `pgTable(` calls; the `shared/schema/` extraction directory adds another 87 domain files with 275 more `pgTable(` calls (there is some deliberate overlap during migration). Combined ceiling: ~694 tables. The largest domain files: `csr-knowledge-db.ts` (49 KB), `project-charter.ts` (38 KB), `qc-schemas.ts` (30 KB), `orchestration.ts` (26 KB), `operating-system.ts` (26 KB), `capa-mdr.ts` (25 KB), `regulatory-atoms.ts` (23 KB), `programs.ts` (21 KB), `ana-intelligence.ts` (21 KB), `living-record-spine.ts` (20 KB).

Row-level multi-tenancy. `organizationId` appears 736 times in `shared/schema.ts` alone. Every governed table is scoped. The RLS story is layered: `establishRequestTenantScope.ts` opens per-request session scope from the verified JWT; `RLS_ENFORCE=on` is fail-closed; `0021_enable_rls_everywhere.sql` is the canonical enablement migration; `053_gcc_rls_policies.sql`, `069_gcc_multitenant_rls_expansion.sql`, `070_gcc_rls_extended_ga.sql` extend it. Tenant-column audits (`0019_tenant_column_audit.sql`) confirm no unscoped writes.

Audit chain. Nine audit tables in the monolith (`auditLogs`, `auditTrail`, `documentAuditTrail`, `deviceAuditTrail`, `auditEvents`, `proofAuditLogs`, `regulatoryAuditLogs`, `sharepoint_audit_log`, `qmsInternalAudits`), plus HMAC seals and hash chains (`002_gcc_audit_immutability.sql`, `054_gcc_part11_audit.sql`, `064_gcc_cognitive_audit_schema.sql`, `20260609_audit_hmac_seal.sql`,

`20260325_ana_kernel_log_immutability_hashchain.sql`), plus signed exports (`server/services/audit/signedAuditExport.ts`) and a chain-integrity sweep job (`server/jobs/auditChainIntegritySweep.ts`).

---

14. Security, identity, and 21 CFR Part 11 compliance

Security is not a section in the docs — it is a set of gates the code cannot skip.

Identity and authentication. `bcryptjs` for password hashing, `jsonwebtoken` for access tokens. Access token lifetime is 24 hours; refresh token lifetime is 7 days. Failed-login lockout at 5 attempts (`LOCKOUT_THRESHOLD = 5`) for 30 minutes (`LOCKOUT_DURATION_MINUTES = 30`). Every authentication event is logged via `auditService.logAction` with an explicit annotation citing 21 CFR Part 11 §11.10(e).

Rate limiting. `express-rate-limit` (in-process) and a Redis-backed distributed limiter (`redisRateLimiter.ts`). A central rate-limit registry provides seven named tiers — `global`, `api`, `ai`, `auth`, `write`, `upload`, `export` — mounted on `/api/auth`, `/api/ai`, `/api/export`, `/api/upload`, `/api/workflow`, `/api/documents`. Login is 10/15min per IP; signup 5/hr; password-reset 5/hr; MFA-verify 10/15min.

MFA. TOTP is a first-party implementation in `server/services/mfaService.ts` — RFC 6238, HMAC-SHA1, 30-second period, 6-digit code, ±1 window, 20-byte secret encrypted at rest with AES-256-GCM. Explicitly no dependency on `speakeasy` or `otplib`. Email-OTP fallback (`emailOtpService.ts`) and backup codes for recovery.

Enterprise auth. Four-step flow in `server/routes/authEnterprise.ts`: check-email → verify-password → verify-mfa → select-organization. Any dev-auth bypass is explicitly removed. SAML SSO through `server/services/saml-provider.ts`, SCIM 2.0 provisioning through `server/routes/scim.ts`, and a dedicated SSO router.

Transport and headers. `helmet` at two configurations with a CSP nonce (there is a unit test for the CSP-nonce specifically). Content-Security-Policy applied on every response.

21 CFR Part 11. A dedicated `server/services/part11/` folder (`signing-authority.ts`, `reverify-signer.ts` + tests), `server/services/compliance/signature-manifestation.ts`, and the ANA-side counterparts (`server/services/ana-ri/part11-governance.ts`, `governed-action-signoff.ts`). The e-signature path is `governed-transmit.ts`. Signed audit export via `server/services/audit/signedAuditExport.ts`. Four golden-journey tests exercise the end-to-end signed submission path per pathway: CER EU-MDR, 510(k) eSTAR, NDA eCTD, IND authoring.

GxP, GDPR, HIPAA-adjacent. GDPR: dedicated runtime-role tests (`tests/db/gdpr-service-runtime-role.dbtest.ts`), redaction rules in `sql/redaction_rules.sql`, and tenant-export flows in `server/services/tenant-export/`. GxP referenced through the audit logger, enterprise security, and structured logging. HIPAA-adjacent controls sit inside audit + tenancy + field-level encryption (`server/services/security/field-encryption.ts`).

Secrets. `.env`, `.env.*`, and `.env.local` are gitignored (`.gitignore` line 13). Only example files (`.env.example`, `.env.beta.example`, and two other explicit `.env.*.example` files) are tracked. There are no keys in the repo.

Tenancy middleware. `server/middleware/` : `establishRequestTenantScope.ts` (per-request Postgres scope opening), `tenantContext.ts` , `orgMembership.ts` , `authBoundary.ts` , `authAdapter.ts` , `moduleEntitlementGate.ts` , `authoringObjectAuthorization.ts` . RLS is enforced fail-closed, per the design document `docs/architecture/RLS_ROUTE_LAYER_SYSTEMIC_FIX_DESIGN.md` .

---

15. The frontend surface, design system, and design governance

The frontend is a governed React application, not a design free-for-all.

The shell. `client/src/concept2cure/v2/V2App.tsx` mounts `client/src/concept2cure/v2/Shell.tsx` , which provides four chrome primitives:

- `Rail` — the primary navigation column with up to 24 rail buttons (governed by `railButtonsMax: 24` in `config/ui-surface-registry.json`).
- `TopBar` — global search, command menu launch, presence, notifications.
- `AnaRail` — the AI copilot rail (up to 7 conversations concurrent, per `ownsConversationMax: 7`).
- `CmdK` — command-menu launch.

Surfaces. Twelve views ship today, all registered as first-class surfaces in `config/ui-surface-registry.json` (`Home` , `Chats` , `Projects` , `Project Landing` , `Communication Center` , `Apps` , `Tasks` , `Review` , `Submission Center` , `Editor` , `Vault`), plus two hosted surfaces (`mdx-host` , `pdev-app`) and five destination surfaces.

Governed design tokens. The canonical token source is `design-system/colors_and_type.css` , referenced by the surface registry. Design CI gates enforce: no hardcoded values, no phantom tokens, no shadowed selectors, only governed components from the registry. The design-reviewer, design-system-auditor, a11y-auditor, motion-auditor, and Part 11 UX auditor agents are wired to run against every PR that touches UI.

The AnA copilot. Real-time chat surface with three effort modes (standard, deep-research, nano-banana); the mode toggle is tested at `client/src/concept2cure/v2/__tests__/anaEffortMode.test.ts` and the guard at `server/middleware/nanoBananaGuard.ts` .

Retired surfaces. The registry also tracks `deleted` surfaces (ZenApp, AnaPersistentPanel, EditorPanel, ProjectWorkspaceShell, GlobalOperatingShell, IndustryWorkspaceShell) so no code path can silently resurrect them. This is a deliberate discipline: we do not just delete UI, we mark it deleted in the source of truth so anyone reading the registry knows the migration is complete.

---

16. The harness: our engineering discipline as a moat

An investor should read `CLAUDE.md` in the root of the repo before making a decision. It is a two-page document that establishes the engineering culture, and it is the single most differentiating asset in the codebase after the kernel.

Rule 0 — one branch. `concept2cure-v2` is the only branch anyone (human, agent, worktree, CI job) may push to. Non-canonical pushes are refused at the pre-push hook. The rule is enforced, and the exception path is narrow (external refs like `dependabot/*` and `revert-*` ; agent-shaped branches are refused

unconditionally). This eliminates an entire category of failure — divergent branches, stale mirrors, "which one is truth?" — that consumes the middle years of most startups.

Rule 1 — migrations re-execute. Every migration is re-executed on every deploy, unconditionally. This is a design choice that trades one thing (a small amount of DB work at deploy) for two things (a) an idempotent, replayable schema definition, and (b) a hard rule that DROPs are not the way to remove things — you amend the creating migration in place, and the CI drop-safety gate blocks the alternative.

Both rules are cultural artifacts codified as enforced gates — `CLAUDE.md` is not a wiki page, it is authoritative, and the hooks cite it. That is how you get an engineering organization that ships fast without breaking regulated data.

The agents. Alongside the engineers, the repo has 14+ specialized reviewer agents wired in: `a11y-auditor`, `design-reviewer`, `design-system-auditor`, `motion-auditor`, `microcopy-reviewer`, `honest-state-auditor`, `part11-ux-auditor`, plus code-focused agents. They run on every UI-touching PR. This is not a demo — this is how we scale review coverage across 439 client files without hiring a design-review team.

The doc discipline. 672 markdown files in 55 subdirectories of `docs/` — architecture, audits, runbooks, roadmap, deployment, release, standards, AI governance, compliance, security. Design decisions are written down. Retirements are written down. Trade-offs are written down. An investor is invited to sample five random docs from `docs/architecture/` or `docs/reports/` to verify.

---

17. Testing, observability, and reliability posture

Automated tests. 2,348 test files across the repo: 1,522 under `server/`, 301 under `client/`, and 448 in the dedicated `tests/` tree that houses integration and end-to-end suites — including `tests/e2e/` (design tokens, submission ops, governed lifecycle, RC-beta path, biotech modules, golden customer journey, beta pulse, submission-ops UI, diff history) and `tests/golden-journeys/` (CER EU-MDR, device 510(k) eSTAR, drug NDA eCTD, IND authoring). Playwright is configured (`playwright.config.ts` at repo root plus `scripts/visual-qa/playwright.mjs`) and the `gstack` QA harness (`.claude/skills/gstack/`) runs headless browser dogfooding of live surfaces.

Observability. Sentry (both `@sentry/node` and `@sentry/react`) for error tracking. Structured logging through `logger.ts`. The AI Gateway emits scoped logs (`createScopedLogger('ai-gateway')`) with per-provider health tracking. Kernel emits observability trace steps on every decision. The audit chain-integrity sweep runs as a scheduled job (`server/jobs/auditChainIntegritySweep.ts`).

Reliability. Bull-on-Redis queue for long-running AI actions (`refine_with_validation`) and exports (`export_document`) with durable retry, exponential backoff, dead-letter queue for failed actions, progress tracking via SSE, horizontal scaling across multiple workers, and graceful drain-on-shutdown. When Redis is unavailable, the queue falls back to synchronous execution rather than failing the request outright. Socket.io provides the real-time channel for collaboration primitives (`CollaboratorInfo`, `CursorPosition`, `BatchUpdateData`) with JWT authentication.

Deployment posture. Neon Postgres, direct-endpoint enforced for migrations. Migrations replayable. `RLS_ENFORCE=on` fail-closed. `.env` files never in the repo. Feature-tier flags via `kernel-adaptive-policy.ts` so the same binary ships to beta and GA.

---

18. The competitive moat, in one page

Ask a diligence reader to list three things a competitor would need to have to be a peer, and it is roughly these:

A governance kernel. Not a chatbot, not a policy config file — a set of composable decision functions that evaluate every governed action against governance, security, and observability domains, emit an immutable decision record, and can be re-derived from source months later. We have this. Competitors selling AI to regulated buyers do not, and the sales pitch that follows an inspection question they cannot answer is short.

A memory system with a supersession lifecycle. Not "chat history," not a vector store, but a three-layer memory (working, client, project) that carries locked facts, decisions, open questions, and next actions, with entries that transition `active → superseded` with successor pointers. This is what turns an AI from a session into a collaborator. Everyone can add short-term memory. Adding memory that is safe to keep across users is the hard part.

A submission twin. A live in-database model of the submission with claim-to-evidence integrity, drift detection, change-consequence intelligence, and readiness modeling. This is the piece that turns "we drafted your Module 2" into "we know your submission is 68% ready, we know which four blockers dominate the risk, and we know which Module 3 spec change will invalidate a Module 5 claim."

Add to that the design system, the harness, the 21 CFR Part 11 wiring already in production, and the 554 versioned migrations, and the picture is a platform that would take a competitor two years to catch, minimum, and by that time we will have moved.

---

19. Roadmap and near-term milestones

Roadmap themes for the four quarters ahead, in the order they will land:

Q1: Deeper regulator coverage. Expand precedent depth for EMA and PMDA. Add German-language Notified Body correspondence to the CER pathway. First production sponsor on the platform across all nine workstreams.

Q2: Advanced twin capabilities. Full change-consequence propagation across Modules 1–5. Real-time regulator challenge simulation surfaced as a live sidecar during authoring. Deeper reviewer-simulation model, tenant-tunable.

Q3: Federated learning and multi-sponsor precedent. With sponsor opt-in, precedent depth increases via de-identified outcome sharing. The federated-learning service (`server/services/cognitive-ecosystem/federated-learning.service.ts`) is already in place; the gating is contractual, not technical.

Q4: Platform extensibility. MDX-host and pdev-app surface types will graduate into a first-class app SDK so sponsors and partners can ship their own governed apps on our kernel. This is the platform play — the same kernel that grants an internal action can grant a partner's action.

Each quarter delivers commercial anchors, not just features. Each anchor is measurable, and each will be reported to investors against the milestone plan.

---

20. Appendix — quantitative inventory

The numbers below are the ones an engineering diligence reader will want in one place. Every figure is grounded in a file path in the repo.

20.1 Codebase size

DIMENSION	COUNT	REFERENCE
Server TypeScript files	4,180	`find server -name '*.ts'`
Client React (`.tsx`) files	439	`find client/src -name '*.tsx'`
HTTP route modules (top level)	368	`server/routes/*.ts`
Service files (recursive)	2,956	`server/services/**/*.ts`
Service subdirectories	222	`server/services/*/`
Kernel service files	8	`server/services/kernel-*.ts`
Intelligence layer modules	37	`server/services/intelligence/*.ts`

20.2 Database

DIMENSION	COUNT	REFERENCE
Migration files total	554	`migrations/` (247) + `db/migrations/` (307)
Baseline migration size	420 KB	`migrations/0000_sweet_joseph.sql`
`shared/schema.ts` size	840 KB	monolith
Domain schema files	87	`shared/schema/`
`pgTable` declarations	~694	419 (monolith) + 275 (domain)
`organizationId` occurrences	736	`shared/schema.ts` alone
Embedding dimensions in use	1024, 1536, 3072	pgvector
Report domains supported	12	`intelligent-report-engine.ts`
Regulatory bodies supported	15	`intelligent-report-engine.ts`

20.3 Versioning of intelligence layer

COMPONENT	VERSION CONSTANT	FILE
RIM	`1.1.0`	`server/services/intelligence/rim.ts` line 79
Judgment framework	`1.2.0`	`judgment-framework.ts` line 39
Pattern registry	`1.3.0`	`pattern-registry.ts` line 26
Judgment models	6	Evidence Sufficiency, Defensibility, Reviewer Sensitivity, Claim Risk, Cross-Section Consistency, Submission Risk
Pattern categories	8	deficiency, reviewer_trigger, rejection, strong_language, weak_language, data_gap, consistency_issue, formatting

20.4 Security

CONTROL	VALUE	SOURCE
Access token lifetime	24 hours	`server/routes/auth.ts:102`
Refresh token lifetime	7 days	`auth.ts:103`
Login lockout threshold	5 failed attempts	`auth-security-service.ts:31`
Login lockout duration	30 minutes	`auth-security-service.ts:32`
MFA algorithm	TOTP RFC 6238 / HMAC-SHA1	`mfaService.ts`
MFA period / digits / window	30s / 6 / ±1	`mfaService.ts`
MFA secret at rest	AES-256-GCM	`mfaService.ts`
Enterprise auth steps	4	`authEnterprise.ts`
SSO / SCIM	SAML + SCIM 2.0	`saml-provider.ts`, `scim.ts`
CSP	helmet + nonce	`enterprise-security.ts:166,213`
Rate-limit tiers	7 named	`enterprise-security.ts:417`
Audit tables	9 in monolith + more	`shared/schema.ts`
Audit immutability	HMAC seal + hash chain	`audit-hmac-seal.ts`, `chainIntegrityMonitor.ts`
RLS enforcement	fail-closed	`RLS_ENFORCE=on`

20.5 Tests and docs

DIMENSION	COUNT
Total test files	2,348
Tests under `server/`	1,522
Tests under `client/`	301
Dedicated `tests/` suite	448
Golden-journey suites	4 (CER-EU-MDR, 510(k) eSTAR, NDA eCTD, IND authoring)
Docs `.md` files	672
Docs subdirectories	55

End of Investor Technical Brief.

Prepared on branch `concept2cure-v2`, the sole production branch, per repository policy. Every metric in this document is verified against source files at the paths cited.
